

Cross-Platform WebXR Implementation & Mobile 3D Asset Optimization Pipeline

Augmented Reality and Mixed Reality (DJS23ALH1501), B.Tech Sem V, AI&ML | Assignment 1 | CO1, CO2

Name: [your name] Roll No: [roll no] Batch: [batch] Live URL: [https://<user>.github.io/<repo>/] Repo: [https://github.com/<user>/<repo>]

Part A - Native SDKs vs browser-based AR: architectural trade-offs (CO1)

The decisive constraint for app-less, cross-platform AR is browser support. The WebXR Device API is available in Chromium browsers (Chrome, Edge, Samsung Internet) and the Quest browser, but Safari on iOS, iPadOS and macOS does not implement it; Safari on visionOS 2 supports it only for immersive-vr (references 1, 2). An iPhone user therefore cannot open a WebXR immersive-ar page. Apple's route to AR from the web is AR Quick Look, a viewer for pre-made USDZ models with no custom logic or tracking. Camera-based JavaScript libraries (MindAR.js, AR.js) avoid this gap because they need only getUserMedia and WebGL, which every modern mobile browser provides. The price is that they track 2D images or markers from camera pixels alone, without the IMU fusion and depth sensing that ARKit and ARCore provide.

Criterion	Apple ARKit (native)	Google ARCore (native)	WebXR Device API	MindAR.js	AR.js
Sensor access	Camera + IMU visual-inertial odometry; LiDAR depth/scene mesh on Pro devices; plane, image, face, body tracking; persistent world maps.	Camera + IMU VIO; Depth API; plane, augmented-image and face tracking; Geospatial and Cloud Anchors.	immersive-ar session backed by ARCore (Chrome/Android): hit-test, anchors, light estimation, optional depth. Raw camera frames are not exposed to JS.	Camera via getUserMedia only. Image and face tracking run in-browser (TensorFlow.js). No IMU fusion, so no world tracking/SLAM.	Camera via getUserMedia. ARToolkit fiducial markers, NFT image tracking, GPS + compass location AR. No SLAM.
Rendering throughput	Metal via RealityKit/SceneKit; typically 60 fps; limited mainly by device GPU.	OpenGL ES/Vulkan via Filament/Sceneview; 30 or 60 fps camera configs.	WebGL/WebGL2 (three.js, Babylon). Near-native for light scenes; JS and WebGL overhead; tight triangle/draw-call budget.	WebGL via three.js/A-Frame, sharing the GPU with the TF.js tracker, so less headroom for the scene.	WebGL via three.js/A-Frame; marker tracker runs in WASM on CPU, so the GPU is mostly free for the scene.
Latency	Lowest: OS-level sensor fusion and pose prediction.	Low: OS-level fusion and prediction.	Low where supported: pose comes from ARCore; adds browser compositor and JS frame.	Higher: each frame passes video, JS tracker and pose smoothing (One-Euro filter trades jitter for lag).	Low for markers (fast WASM detector); noticeably higher for NFT natural-image targets.
Memory footprint	App binary typically tens of MB plus assets; direct GPU memory control.	Same as ARKit, plus ARCore services on device.	No install; the browser tab is memory-capped and can be killed under pressure.	A few MB of JS (bundles TF.js) plus a small .mind file; tab memory-capped.	Smaller JS than MindAR; tab memory-capped.
Onboarding friction	Highest: store search, download, install, permissions (about 5 steps). iOS only.	Highest: store install; device must support ARCore (about 5 steps). Android only.	Low: open URL, grant permission, start AR (about 3 steps). Chromium browsers on Android only; no iOS Safari support.	Lowest: scan QR / open URL, grant camera (about 2 steps). Any modern iOS or Android browser over HTTPS.	Lowest: same as MindAR.
Best fit	Premium iOS experiences needing LiDAR or persistence.	Android apps needing depth, geospatial or anchors.	Android browser AR with surface placement.	Cross-platform image or face tracking with no install.	Fiducial markers and location AR with the lightest footprint.

Table 1. Comparative analysis. Ratings are qualitative and based on the vendor/library documentation and cited sources; step counts and sizes are typical values, not measurements made for this assignment.

Trade-off summary. Native SDKs win on tracking quality, latency and throughput because they read the IMU, depth sensors and GPU directly; the cost is two codebases, store distribution and the highest onboarding friction. WebXR gives near-native pose tracking with zero install, but only on Android Chrome. MindAR.js and AR.js give the widest reach and lowest friction, with weaker tracking (image-anchored only), higher latency, and a tab that must stay inside browser memory limits. These limits make 3D asset size the main lever a developer controls, which motivates Part B. For this project the target is a printed image shown to any phone, so **MindAR.js + A-Frame (three.js)** was chosen: it provides natural-image tracking on both iOS and Android without installation.

Part B - 3D asset decimation and pipeline optimization (CO2)

Source asset. An ornamental vase (diamond-stud belly, neck beads, foot and shoulder ridges) was authored procedurally as a surface of revolution with relief detail: 737,280 triangles, 21.19 MB. The pipeline (pipeline/make_asset.py) is scripted so every number below is reproducible; its stages correspond one-to-one to the Blender operations in the brief (a Blender bpy script is included in pipeline/blender_pipeline.py).

- **Decimation.** Vertex clustering over 12 x 10 grid cells (equivalent to Blender's Decimate modifier at ratio about 0.0083) reduces the mesh by a factor of 120 to 6,144 triangles; the pole rows are re-closed so the vase stays watertight.
- **Baking.** A 1024 x 1024 tangent-space normal map (PNG) and an albedo map with baked cavity/AO (JPEG) are baked from the high-poly surface onto the low-poly UVs. Tangents are exported with the mesh (glTF TANGENT, w = +1), so the renderer reconstructs the same TBN frame used for baking.

- **Merge.** The three material groups (foot, body, neck/inside) collapse into one mesh with one material, cutting draw calls from 3 to 1.
- **Export.** Binary glTF 2.0 (.glb) with embedded textures. Draco compression is the last step (optimize_draco.sh) and is loaded in the page through A-Frame's dracoDecoderPath.



Figure 1. Software renders of the exported .glb files. (b) without baked maps the relief is lost; (c) with baked maps at 0.8% of the triangles the lattice, beads and ridges are preserved.

Metric	Before (high-poly)	After (decimated + baked)	After + Draco (fill in)
Triangles	737,280	6,144 (-99.17%)	unchanged
Vertices	369,889	3,185	unchanged
Draw calls (primitives / materials)	3 / 3	1 / 1	1 / 1
Textures	none (vertex colours)	2 x 1024x1024 (diffuse JPEG, normal PNG)	same
File size (MB)	21.19	0.65 (-96.9%)	run optimize_draco.sh
Estimated GPU memory (MB)	21.1	10.8	run optimize_draco.sh

Table 2. Before/after metrics measured from the exported files (triangles and size are exact; GPU memory is an estimate: vertex/index buffers plus mip-mapped RGBA textures). The yellow column is filled in after running the Draco step.

Findings. Triangles fall 99.17% and file size 96.9% (21.19 MB to 0.65 MB). Of the optimized file, the textures account for about 71% (0.46 MB) and geometry for about 29%. Draco compresses only the geometry, so its gain here is limited to that remaining share; further savings must come from texture compression (WebP or KTX2/Basis). The limitation of baking is the silhouette: the relief (about 4% of the belly radius) is no longer geometry, so it appears in shading but not in the outline, which is acceptable for a small AR object.

Part C - Mini-demonstration deployment (CO2)

The page (web/index.html) loads pinned A-Frame 1.5.0 and MindAR 1.2.5, tracks a compiled image target (targets.mind, built from target.png with the MindAR compiler), and anchors the optimized .glb on it: the model is scaled to about half the target's height, rotated 90 degrees so it stands on the card, and slowly spun. Tracking smoothing is set through missTolerance/warmupTolerance. The site is static, so it is hosted on GitHub Pages, which supplies the HTTPS that mobile browsers require for camera access. Opening the URL with ?debug=1 shows A-Frame's stats panel (fps, triangles, draw calls) and logs per-second FPS to the console; ?model=vase_optimized_draco.glb swaps in the Draco build.

Device	OS / browser	Avg FPS	Tracking lock time / stability	Notes
[e.g. Android phone]	[Chrome 1xx]	[]	[]	[]
[e.g. iPhone]	[Safari 1x]	[]	[]	[]
[optional 3rd device]	[]	[]	[]	[]

Table 3. On-device verification (to be completed after deployment). Record average FPS over at least 30 s, time-to-lock, and whether the model stays anchored under hand movement and moderate viewing angles.

Conclusion

Browser AR trades tracking sophistication for reach and zero install friction; native SDKs are the reverse. Because the browser tab is the constraint, a disciplined asset pipeline (decimate, bake, merge, compress) is what makes browser AR viable: the optimized vase draws in one call with 0.8% of the original triangles while retaining its visual detail.

References

[1] TestMu AI, "WebXR Browser Support in 2026: What Works, What Breaks" - <https://www.testmuai.com/learning-hub/webxr-compatible-browsers/>

[2] W3C Immersive Web WG, WebXR Device API (Candidate Recommendation Draft, 9 June 2026) - <https://www.w3.org/TR/2026/CRD-webxr-20260609/>
[3] MindAR documentation and compiler - <https://hiukim.github.io/mind-ar-js-doc> | AR.js - <https://ar-js-org.github.io/AR.js-Docs/>
[4] A-Frame 1.5 docs (glTF-model, Draco) - <https://aframe.io/docs/1.5.0/> | Draco - <https://google.github.io/draco/> | glTF 2.0 - <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>